

alterspec: the essential guide

How a product spec is started, tuned, designed, changed and handed over with alterspec, and which parts of that work are deterministic (the CLI) and which are done by the model (Claude Code skills and agents).

1. What alterspec is and how it thinks

alterspec is a product specification framework for teams that deliver with AI. Its name is its thesis: the spec changes with the product. Code has become cheap to produce, so the scarce thing is a clear, current, agreed description of what the product does. alterspec keeps that description as one source of truth and makes every change to it reviewed, traceable and checkable, by people and by machines alike.

What the spec is. A folder of Markdown files with YAML front-matter, `spec/`, that describes the product in business language only. Three levels carry it: the **Application** (what the product is, for whom, its personas, roles, glossary, entities, shared rules, flows, events, integrations, quality requirements and decisions), its **Modules** (business areas, each with its own rules), and their **Capabilities** (one user goal reached in one session, with the roles allowed to perform it and testable acceptance criteria). Screens say what people see and do; entities say what the business things are and how they live. Nothing in it names a database, an endpoint, a framework or a library. alterspec is a spec-driven framework that covers only the product spec; the technical plan and tasks belong to the development team. It delivers the spec to that team as a self-contained bundle, and never acquires a technology layer of its own.

The principles that shape every command:

- **Front-matter is authoritative, prose explains.** Who may do what, which screens show which data, which entity transitions a capability performs: all of it is structured data. Everything that can be derived from it (module capability lists, role matrices, back-references, coverage, traceability, the wireframe, the data the mockups load) is generated into marked blocks and never written by hand. A hand edit of a generated block is an error.
- **Stable identity.** Every object has an ID that is never renumbered or reused. Every reference is checked to exist. Capabilities, screens, entities, flows, rules and events form closed loops the linter enforces: an action on a screen is a capability one of its roles may perform, a shown field is an attribute of the entity, a consumed event has an emitter, a lifecycle transition is performed by someone.
- **Ask, don't invent.** The model interviews, at most three questions at a time, and drafts from the answers only. What nobody knows yet becomes an open question (DEC-*) attached to the object, and the object cannot advance until it is answered. Status is earned: `draft` -> `refined` -> `ready` -> `approved` -> `implemented`, where `refine` may reach `refined` on its own and everything beyond needs the person's explicit word.
- **Deterministic core, model at the edges.** The CLI owns IDs, file locations, templates, validation, generation, fingerprints, merging and handoff rendering; the same input always gives the same output, and all of it runs in CI without Claude. The skills and agents own conversation and judgement: interviewing, drafting, semantic review, shaping a design. The model never picks an ID, never copies a template, never approves a change.
- **Change by proposal.** Until the first version is agreed, you edit freely. Then `baseline` records a fingerprint of every object, and from that moment the main spec is read-only for people. Every edit is a change proposal (CHG-NNN) holding copies of only the touched objects, validated as the spec it would produce, reviewed for impact, approved by a person naming the change, and merged by the CLI. Think of it as version control at the level of product decisions rather than lines.
- **The UI is a projection of the spec, not a second spec.** A generic wireframe is generated from the business spec and is always current. The experience layer, a working mockup app, realises exactly what that wireframe shows, element for element, and the checks allow zero deviation. Developers receive a screen only when its experience is reviewed and clean.

Who it is for. A product owner or business analyst working with Claude Code writes and changes the spec; the reviewer agents and the linter keep them honest; developers and their delivery frameworks receive handoffs with exact versions. The open question recorded in the project is whether the analyst leads and the AI interviews, or the AI drafts and the human approves. The commands are built interview-then-draft; a different answer changes their design, not the spec model.

2. Starting a new project

- 1 In the project root: `npm install --save-dev @alterset/alterspec` then `npx alterspec init --name "My Product"`. This writes three folders, all meant to be committed: `spec/` (yours), `.alterspec/` (templates, prompts, schemas, config, your `custom/` overrides) and `.claude/` (the `/alterspec-*` skills and the four agents).
- 2 Open Claude Code in the project and type `/alterspec-init`. Claude interviews you, at most three questions at a time: the product and its problems, the product profile, personas and roles, the business areas (modules, each with a 2–6 letter code; shared screens go in GLB), and the key terms with the words not to use. The **product profile** narrows the spec effort before it starts: which apps and channels exist (and whether a web channel must work on phones), one company or many, the languages and currencies (with a mandatory default when there are several) and the time zones. It lands in `application.md` through `alterspec new channel` and `alterspec profile set`. Every later interview reads it and doesn't ask about what it settles; the linter reports currency conversion, translation, tenants or time zones in the spec when the profile rules them out, so nobody specifies a multi-currency product that is single-currency.
- 3 It then writes `spec/application/`: `application`, `personas` and `roles`, `glossary`, and one `module.md` per module. Every object is created by the CLI (`alterspec new ...`), which picks the ID, the file and the template.
- 4 Finish with `/alterspec-validate`. The spec is readable Markdown with YAML front-matter; you can edit it by hand at any time, the linter tells you when something no longer fits.

What exists after this step: the skeleton. No entities, capabilities or screens yet.

3. Talking to alterspec

Once the skeleton exists you don't need to know commands. Open Claude Code in the project and say what you want, in the product's own words. What happens depends on what you said, and you don't pick the path:

- **A question.** "What can a sales rep do with a price?" "Is pricing ready for developers?" "What is still open?" Claude answers from the spec. Nothing is changed.
- **A small change, one thing.** "Add gender to buyer." "The approval screen should also show the margin." Claude shows you the object it understood, asks the two or three things it needs (kind, options, required, who sees it), makes the edit, and tells you which other objects it touches. After the baseline this happens inside a change proposal, and you'll be told its ID.
- **Something new.** "We need a Supplier." "Buyers should be able to reorder." Claude creates the entity or the capability where it belongs, interviewing you first.
- **A feature.** "Buyers should see their order history and reorder from it." Claude opens a change proposal and works through every object it needs, asking as it goes. (A later version will draft the whole proposal first and ask you once.)
- **Looks and behaviour.** "Make the order history a table, newest first." That is the experience layer; the business spec is not touched.

If a sentence could mean two things, Claude asks one question before doing anything. `/alterspec` followed by your words does the same routing explicitly, and `/alterspec` alone answers questions about the product.

What you still type yourself, because each one is a decision only you can take: `/alterspec-init` once at the start, `npx alterspec baseline` when the first version is agreed, `/alterspec-apply CHG-...` with your

approval, and `/alterspec-handoff CAP-...` to give a capability to the developers.

4. Tuning the spec during initiation

The first version is built before any baseline exists, so edits go straight into `spec/`.

- `/alterspec-create-entity NAME Title`: a business thing with attributes (business kinds only: text, amount, date, choice, reference...), relationships and a lifecycle (states and allowed transitions).
- `/alterspec-create-capability MOD "Title"`: one user goal, one session, the roles allowed to perform it. The interview walks through the thirteen sections (user story, value, preconditions, main flow per screen and action, exceptions, data in and out, rules, state transitions, notifications, permissions, acceptance criteria, out of scope, open questions). The analyst agent drafts the body from your answers only; it never invents facts.
- `/alterspec-create-screen MOD "Title"`: what people see and do, in business terms. `fields` lists the data shown per entity (attribute names exactly as the entity has them, in `list`, `view` or `edit` mode); `actions` name the capability each performs.
- `/alterspec-refine ID`: closes the gaps in one object. It merges the linter's findings with the analyst's gap analysis, asks only about those, and moves the object to `refined` when nothing is missing. `ready` and `beyond` always need your explicit word.
- `/alterspec-validate`: the 45 deterministic rules plus the reviewer agent's semantic findings (contradictions, data nobody produces, permission holes, untestable criteria) in one report. Fix errors as they appear; it is far cheaper than later.

Tuning never means guessing: when you don't know yet, the answer is recorded as an open question (`DEC-*`) against the object, and the object cannot become `refined` until it is answered.

When the first version is agreed: `npx alterspec baseline`. From then on every edit is a change proposal (section 7).

5. Wireframe and experience: making and tuning the UI

Two layers show the product. Think of them as a dry and a wet signal.

Dry: the generic wireframe. `alterspec views` writes `spec/_generated/wireframe/`: one plain page per screen, built only from the business spec (fields, actions, roles, entry points, the empty / no-permission / validation states) with made-up data. Anything the spec doesn't say shows as a highlighted "Not specified" note. You never edit it; it is always current, and it is what the experience layer is measured against.

Wet: the experience layer (`spec/experience/`), the future app:

- `design-system.md` (tokens, components), `patterns.md` (page archetypes and their regions).
- `screens/UX-SCR-...md`: the UX contract of one screen. Every business element with its region, component and exact label; every state as a link (`?as=<role>&state=<id>`); interactions, validation messages, loading, errors, responsive and accessibility behaviour.
- `mockups/`: a working app. `index.html` signs you in as a demo person; `SCR-...html` pages are bound to demo data kept in the browser (lists open records, forms save, actions change data the way their capability says); `config.js` (name, logo, currency, demo people) and `data.js` (demo data) are yours; `kit/` is the runtime.

How you change the UI:

- 1 `/alterspec-experience init` once: the app, its name and brand, demo people, realistic data.
- 2 `/alterspec-experience SCR-XX-NN` per screen: the CLI drafts the contract and the page so that they already pass every check; you answer what the draft cannot know (archetype, placement, components, labels, what happens after each action); the UX designer agent shapes both files. Open `spec/experience/mockups/index.html`, sign in as each role, try it, iterate.

- 3 `/alterspec-experience review SCR-XX-NN`: the experience reviewer agent produces a parity table (business spec, contract, mockup, verdict per element). Only a clean table lets `experience reviewed` record the review.
- 4 Edit the page or the contract by hand whenever you like. Keep `data-src`, `data-roles` and `data-show-in`; the checks read them. Design-only elements (icons, separators, help text that restates the spec) are free. Your edits are never overwritten: `rebuild` replaces a page only while it is the untouched draft; afterwards it keeps the page, writes the fresh render to `_generated/experience/rebuild/` and reports what differs: business content the spec lacks (goes through the change flow), check findings, or an outdated kit to merge by hand.

How everything stays in sync:

- Every business element of a screen has a stable marker, `data-src="SCR-PRC-01.A01"`. The contract must list each marker exactly once and nothing else; the page must carry each marker exactly once and nothing else; the text inside must contain the declared label; narrowed elements must carry the right `data-roles`; every declared state must be marked `data-show-in`. These are lint errors, not warnings.
- Each contract records the fingerprint of the dry page it was aligned with. When the business screen changes, the linter warns `experience-stale`; `alterspec experience sync SCR-...` removes dropped elements, adds new ones as drafts to place, adds states and default views for new roles, updates who sees what, and records the new alignment. Each contract also records the fingerprint of itself and its page at the last clean review; any later edit warns `experience-unreviewed`.
- If the design needs something the business spec lacks (a field, a filter, a message), it goes into the business spec first, then `sync`. Never only into the mockup: the checks would reject it.

6. Why spec, wireframe and experience cannot drift apart

- One source of truth: capability, screen and entity front-matter. Module lists, role matrices, screen back-references, entity coverage, traceability, the wireframe and the data the mockups load (`spec.js`) are all generated from it. Generated blocks carry a hash; a hand edit is an error.
- One vocabulary: IDs are stable and never reused; every reference is checked to exist; glossary synonyms and technology words are flagged.
- Closed loops: every entity transition is performed by some capability; every consumed event has an emitter; every flow step is a capability and every capability is in a flow; every screen action is a capability one of the screen's roles can perform; edited data on a screen is created or updated by one of its actions.
- The experience layer realises exactly the dry page (section 5), and a business screen can't go to review in a change until its experience follows.
- Handoff refuses anything with lint errors, any capability below `ready` (unless you say `--allow-draft`), and any screen whose experience is not ready, reviewed and finding-free. There is no override for the last one: developers only ever receive aligned screens.

7. Changing an existing feature

After the baseline, the main `spec/` is read-only for people. Direct edits are `direct-edit` errors.

- 1 Start or pick a change: `/alterspec-change "Why we change this"`, or let `/alterspec-change-capability CAP-...` (`-screen`, `-entity`, `-module`) handle one object and ask you which change to use.
- 2 Every touched object is copied into `spec/changes/CHG-NNN/spec/` at its normal path (`alterspec change edit`), new objects are created there (`alterspec new ... --change CHG-NNN`), removals are recorded (`alterspec change remove`). The copy remembers the fingerprint it started from.

- 3 Work inside the change: edit the copies, run `alterspec validate --change CHG-NNN` (the spec as it would be after the change), `alterspec change sync CHG-NNN` so the experience of every affected screen follows (Claude runs it for you and tells you what to place and review), and `/alterspec-impact CHG-NNN` for what is affected: flows, acceptance criteria, screens, roles, generated views, new and resolved findings, plus the reviewer agent's semantic findings.
- 4 `alterspec change status CHG-NNN in_review` is refused while the change has errors, conflicts or no content.
- 5 `/alterspec-apply CHG-NNN` shows the impact and asks for your approval by ID ("approve CHG-NNN"). Nothing else counts. Then it merges, raises `version` on every modified object, regenerates views, updates the baseline and archives the proposal.

Two changes in flight never get the same new ID. If one is applied and touches an object the other also edits, the second gets a conflict and must re-read the object and `change edit --rebase`. A change edited after approval has to be approved again.

8. Creating a new feature

A new feature is a change proposal that mostly adds:

- 1 `/alterspec-change "New feature";` describe why and what, in business language.
- 2 Inside it, the same commands as during initiation, all with `--change CHG-NNN` behind the scenes: new entity or attributes, new capabilities (one goal, one session each), new or changed screens, rules, events, a flow step that ties the feature into an existing journey.
- 3 `/alterspec-refine` the new objects until they are refined; `/alterspec-experience` their screens.
- 4 `/alterspec-impact`, `review`, `in_review`, `/alterspec-apply` with your explicit approval.
- 5 When it is ready and its screens are reviewed: `/alterspec-handoff CAP-...` exports a self-contained bundle (the spec it needs, the wireframe and the experience of its screens) into `handoff/bundle/<ID>/`, with a manifest of the exact versions handed over.

Before the baseline, the same feature is simply created directly, without a change.

9. Manual changes to the experience, and how they reach the spec

Information flows one way: business spec -> wireframe -> experience. Nothing climbs back automatically. What `alterspec` does instead is notice every hand edit, classify it, and refuse to let business content live only in a mockup. If an edit means the product itself changed, you carry it up the spec yourself, one level at a time, and the checks tell you how far you have to go.

What happens the moment you edit a page or a contract by hand.

- The contract (UX-SCR-...), the page (SCR-...html), `config.js`, `data.js` and the kit are spec objects with fingerprints. After the baseline, an edit in the main `spec/experience/` is a `direct-edit` error like any other. The edit belongs in a change: `alterspec change edit CHG-NNN UX-SCR-XX-NN` copies the contract and its page together into `spec/changes/CHG-NNN/spec/experience/`.
- The contract records the fingerprint of itself and its page at the last clean parity review. Any edit after that turns on `experience-unreviewed`; the screen cannot be handed over until `/alterspec-experience review` passes again and `experience reviewed` records the new fingerprint.
- `alterspec experience rebuild SCR-XX-NN` on a hand-edited page keeps the page and writes the CLI's fresh render to `_generated/experience/rebuild/`. Its result sorts the difference into three bins: `businessChange` (markers the business spec doesn't have), `findings` (contract or page checks) and `kitOutdated` (runtime to merge). Nothing in any bin means the page carries design edits only.

The three kinds of edit, and where each one lands.

- 1 **Design only.** Layout, components, icons, spacing, wording of labels and messages, demo data, app name and brand. These stay in the experience layer: labels and components are decided in the contract, the page follows it. Change touches `UX-SCR-...` and the page; re-review; apply. The business spec is not opened.
- 2 **Visibility and wording of something the spec already has.** An element one role should not get is set `unavailable: hidden` or `unavailable: disabled` with a reason in the contract and `data-roles` on the page. Still the experience layer, because who may perform the action is already in the capability's permissions; the design only decides how the refusal looks. If the roles themselves are wrong, that is kind 3.
- 3 **Business content.** A field that is not in the screen's `fields`, an action that is not in its `actions`, a filter, a state, a message that encodes a rule, a role that does not exist. The checks reject it on the page (experience-mockup: the marker is not an element of the screen; experience-elements on the contract) or the reviewer's parity table names it as "mockup shows, spec lacks". It must go into the business spec first, then `experience sync` brings it down again. The designer agent is forbidden to touch `application/` and `modules/` for exactly this reason.

Carrying a kind-3 edit up, level by level. Work inside the same change; each step is a checked human edit, and the next lint error or the reviewer tells you whether another level is needed.

Level	What the design needed	Where it goes, and what checks it
Screen SCR-...	a shown field, an action, a filter, an empty / error state	<code>/alterspec-change-screen:</code> <code>fields, actions, states.</code> <code>screen-field-attribute</code> demands the attribute exists; <code>screen-role-action</code> demands the action is a capability one of the screen's roles may perform.
Entity ENT-...	the field is not an attribute; a new lifecycle state	<code>/alterspec-change-entity:</code> <code>attribute (business kind, options, references), lifecycle.</code> <code>transition-coverage</code> demands some capability performs every new transition.
Capability CAP-...	the action has no capability; a new exception, rule, permission, AC	<code>/alterspec-change-capability</code> or <code>/alterspec-create-capability --change:</code> roles and scopes, screens, entities, transitions, acceptance criteria. <code>capability-without-flow</code> asks for a flow step.
Module MOD-...	a module-own rule; a new capability or screen in the area	<code>modules/<mod>/rules.md</code> (<code>RULE-<MOD>-NNN</code>); <code>module.md</code> itself is not edited, its capability list, role matrix and screens are regenerated by <code>views</code> . <code>foreign-module-rule</code> warns when a capability borrows another module's rule.
Application	a new role or scope, a term, a shared rule, an event, an integration, a flow step, a quality requirement, a decision	<code>personas-roles.md</code> , <code>glossary.md</code> , <code>rules.md</code> (<code>RULE-NNN</code>), <code>events.md</code> , <code>integrations.md</code> , <code>flows/FLOW-...</code> , <code>nfr.md</code> , <code>decisions.md</code> . <code>unknown-reference</code> and <code>glossary-forbidden</code> connect them to the lower levels.
<code>application.md</code>	the product's scope, apps or channels, or the problem it solves	Edited only when a design change really changed what the product is. Rare, and a sign the change should have started as <code>/alterspec-change "why"</code> rather than on a page.

Most UI-born changes stop at the screen or the entity. The change-object skills ask for the knock-on edits explicitly (the "Referenced by" list of `alterspec show`), and `/alterspec-impact CHG-NNN` lists every object that references what you changed, the flows affected, the generated views that will move, and the findings the change introduces or resolves. The reviewer agent adds what no rule can see: a contradiction with a rule, a permission hole, data the new field needs that nobody produces.

Bringing it back down and closing the loop.

- 1 `alterspec change sync CHG-NNN` (or `experience sync SCR-XX-NN --change CHG-NNN` for one screen): the contract drops elements the screen no longer has, receives the new ones as drafts, gains states and default views for new roles, and records the new alignment with the dry page (clearing `experience-stale`). The page gets the new elements in an "Added by sync" block.
- 2 The designer agent places them where you wanted them in the first place; now they carry a `data-src` the spec recognises.
- 3 `/alterspec-experience review SCR-XX-NN` until the parity table is clean; `experience reviewed` records it.
- 4 `/alterspec-impact, in_review, /alterspec-apply` with your word. Apply merges every copy, raises `version` on each modified object (the contract, the screen, the entity, the capability, `application.md` if it was touched), regenerates the module blocks, matrices, wireframe and `spec.js`, updates the baseline and archives the proposal. The page you designed is kept exactly as you left it; `rebuild` will not replace it.

What cannot happen along the way: a page with business content the spec lacks reaching developers (the handoff gate has no override), `rebuild --force` erasing a design without the person saying so, or an agent editing the business spec from inside a design session.

10. Deterministic parts vs. model parts

Deterministic (the CLI, same input -> same output)	Model (Claude Code skills and agents)
IDs, file locations, templates (new)	Interviewing you, three questions at a time
Schema and the 45 lint rules (<code>validate</code>)	Semantic review: contradictions, gaps, permission holes (reviewer agent)
Generated blocks, matrices, traceability, wireframe, <code>spec.js</code> (views)	Drafting body text from your answers (analyst agent)
Fingerprints, baseline, conflicts, approval hash, merge, versions (<code>change, impact, apply</code>)	Writing the "Why" and "What changes" of a proposal
Experience drafts, sync, dry and review fingerprints, parity checks	Shaping layout, components, labels, realistic data (UX designer agent)
Handoff rendering and the experience gate	Parity table and developer questions (experience reviewer agent)
Doctor, install, update	Optional polish of handoff wording (never content)

Rules that keep the two apart: the model never picks an ID, copies a template, edits a GENERATED block or runs `baseline` or `apply` on its own. Claude may start an authoring skill from your words (that is how section 3 works); it may never start `init`, `baseline`, `apply` or `handoff` that way, those you type. Approval of a change is always your explicit word naming the change. Reviewer agents are read-only by construction (no Write or Edit tools). Everything deterministic can run in CI (`validate, views --check`) without Claude.

11. Updating alterspec

```
npm install --save-dev @alterset/alterspec@latest
npx alterspec update           # refreshes .alterspec/ and the .claude/ commands; never touches spec/, co
npx alterspec views           # regenerates derived output; 0.6 moved the wireframe from _generated/prot
npx alterspec doctor
```

Commands you edited under `.claude/` are skipped by update unless you pass `--force`; customisations belong in `.alterspec/custom/` instead, where updates never reach.

Quick reference

Start	<code>npx alterspec init --name "X" /alterspec-init</code>	<code>npx alterspec profile set show</code>
Author	<code>/alterspec-create-entity capability screen module</code>	<code>/alterspec-refine ID</code>
Check	<code>/alterspec-validate</code>	<code>npx alterspec validate [--json] [--change CHG] npx alterspec view</code>
Design	<code>/alterspec-experience init SCR review SCR sync SCR rebuild SCR</code>	
Freeze	<code>npx alterspec baseline</code>	
Change	<code>/alterspec-change "title" /alterspec-change-capability CAP</code>	<code>/alterspec-impact CHG /alter</code>
Deliver	<code>/alterspec-handoff CAP MOD</code>	
Health	<code>npx alterspec doctor</code>	<code>npx alterspec show ID</code>
Update	<code>npm i -D @alterset/alterspec@latest && npx alterspec update && npx alterspec views</code>	